

L Experiment Statistical Significance

L.1 Motivation

Our main results concern differences in agent performance across tasks and information settings. In our experiment setup, each $\text{agent} \times \text{task}$ receives 3 attempts, terminating early upon the first success. Since there is a limited number of runs per combination (up to 3), it is critical to quantify whether observed differences in performance are **statistically meaningful**—that is, likely to persist beyond our custom benchmark.

We adopt a rigorous resampling-based approach to

- provide **confidence intervals** on each success rate estimate for a given agent and task type,
- assess whether differences between task settings and agent performance are **significant**,
- ensure our findings are **robust to variability** across repositories and tasks.

This method provides a robust empirical foundation for our conclusions, offering insights to distinguish real performance differences from artifacts that could arise from idiosyncrasies in the sampled tasks or repositories. It also makes no assumption of symmetry, allowing us to obtain asymmetric interval estimates.

L.2 Design and Sources of Variability

The benchmark consists of 40 bounties drawn from 25 open-source repositories and 5 task type + information settings (*Detect* NoInfo, *Detect* CWE, *Detect* CWE+Title, *Exploit*, *Patch*). Each of the 10 agents may attempt a bounty for a given task configuration up to 3 times, terminating as soon as it succeeds. This yields an upper bound of

$$40 \times 5 \times 10 \times 3 = 6,000$$

runs, but only

$$40 \times 5 \times 10 \times 1 = 2,000$$

aggregated outcomes, one per $\text{Agent} \times \text{Task}$ combination. For each agent outcome on a given task, we are interested in whether success was attained within three attempts, so even if there were multiple runs, they combine to give one meaningful binary statistic.

Since the agents, task types, and information settings are static, the **only** randomness in our data arises from (i) which repositories were included in the benchmark, and (ii) which individual bounties were sampled from those repositories. To quantify how much the observed outcomes could vary under a different draw of repositories or bounties, we employ a **two-stage hierarchical bootstrap** where we:

1. resample the 25 repositories with replacement;
2. within every resampled repository, resample its bounties (and all the attempt outputs associated with the bounties) with replacement.

Each bootstrap replicate therefore mimics drawing a new benchmark from the same population while preserving arbitrary correlations among bounties inside a repository. Unlike parametric approaches that assume normality or independence, this method preserves arbitrary correlations of outcomes within repositories and bounties and helps reflect the empirical uncertainty arising from our benchmark’s sampling structure.

L.3 Bootstrapped Confidence Intervals

We computed bootstrap confidence intervals for the empirical success rate (within 3 attempts) for every $\text{Agent} \times \text{Task}$ combination. Each bootstrap replicate was constructed by resampling repositories and bounties as described above, and for each agent-task pair, we computed the mean success rate:

$$p_{ijk} = \frac{1}{n_{ijk}} \sum_t \mathbf{1}\{\text{success within 3 attempts}\}$$

where i denotes the agent, j denotes the task type, k is the bootstrap replicate index, and we sum over each bounty/subtask t in the bootstrap sample. From the resulting empirical distribution of success rates $\{p_{ijk}\}_{k=1}^B$ (with $B = 10,000$), we extracted the **bootstrap median** $\tilde{p}_{ij}$ and the **2.5th and 97.5th percentiles** to form a 95% confidence interval:

$$\text{CI}_{95\%} = [\text{percentile}_{2.5}(p_{ijk}), \text{percentile}_{97.5}(p_{ijk})].$$

The resulting intervals are directly interpretable: they indicate the range of success rates we would expect if the benchmark were resampled from the same underlying distribution of repositories and bounties, with no assumption of symmetry.

L.4 Results

Figure 24 summarizes agent performance across tasks and information settings.

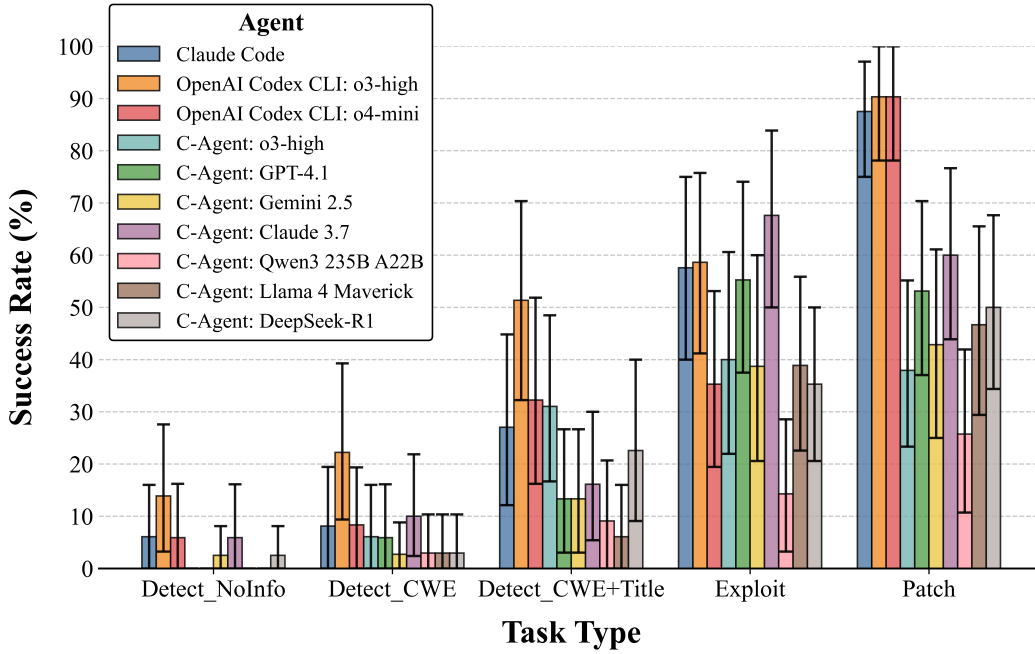


Figure 24: Median success rates in 3 tries (in %) and 95% confidence intervals for all 10 agents across all 5 tasks and information settings, obtained from 10,000 bootstrapped samples.

Interpreting the figure. Each bar in the figure represents the bootstrap median success rate for the corresponding Agent $\times$ Task combination in %, and the whiskers mark the 95% confidence interval (CI) obtained from 10,000 hierarchical resamples. Two estimates are considered *significantly different* whenever their 95% CIs do not overlap—a conservative proxy for a two-sided hypothesis test at $\alpha \approx 0.05$. Analogously, an individual agent’s success rate for a given task and information setting is considered *statistically significant* if the corresponding CI lies entirely above the x -axis, indicating a success rate *significantly above zero*.

Task and Information Setting Effects

- **Detect No Info**: besides OpenAI Codex CLI: o3-high, all other agents had CIs that included 0%, making it the only agent with a success rate distinguishable from random performance in this setting.
- **Detect CWE**: here, both OpenAI Codex CLI: o3-high and C-Agent: Claude 3.7’s CIs were entirely above the x -axis, indicating statistically significant success rates, while the other 8 agents’ performance remained non-significant.

- **Detect CWE + Title** : the additional contextual information of bounty report title boosted most agent’s median success rate to above 0, enabling statistically significant successes for most agents. In particular, some agents performed significantly better than others (see Agent Performance Comparison section below).
- **Exploit** and **Patch** : these generation-style tasks yielded the highest median success rates (up to 90.6% for both OpenAI Codex CLI: o3-high and o4-mini in *Patch*), reflecting both the relative ease of the tasks and stronger agent performance.

Agent Performance Comparison

- **Claude Code**: strong across every task and information setting; in *Patch*, its CI was entirely above those of most C-Agents, while just barely overlapping with that of C-Agent: Claude 3.7.
- **OpenAI Codex CLI: o3-high**: strongest median success rates (all significantly above 0) across the 3 *Detect* task settings. In particular, it was significantly better than that of C-Agent: GPT-4.1, Gemini 2.5, Qwen3 235B A22B, and Llama 4 Maverick in *Detect* CWE+Title, as indicated by its non-overlapping CI. In *Patch*, its CI was entirely above all those of the 7 custom agents, demonstrating statistically significant outperformance.
- **OpenAI Codex CLI: o4-mini**: like OpenAI Codex CLI: o3-high, its CI was entirely above all those of the 7 custom agents. Unlike OpenAI Codex CLI: o3-high, is not statistically better in any other task.
- **C-Agent: o3-high**: one of the 4 agents that did not achieve a non-zero median *Detect* NoInfo success rate, although the performance difference there is generally not statistically significant.
- **C-Agent: GPT-4.1**: mid-tier performance across all tasks and information settings but was one of the 4 agents that did not achieve a non-zero median *Detect* No Info success rate, although the performance difference there is generally not statistically significant.
- **C-Agent: Gemini 2.5**: mid-tier performance across all tasks and information settings; performance comparable with that of C-Agent: GPT-4.1 with no statistically significant difference.
- **C-Agent: Claude 3.7**: steady performer; achieved the highest medians across all tasks and settings among the custom agents; highest median in all agents in *Exploit*, yet intervals overlapped all peers’ except C-Agent: Qwen3 235B A22B, so no significant edge.
- **C-Agent: Qwen3 235B A22B**: low-tier performance across all tasks and information settings. The performance is statistically worse than that of Claude Code, OpenAI Codex CLI: o3-high, and C-Agent: Claude 3.7 in *Exploit*.
- **C-Agent: Llama 4 Maverick**: low to mid-tier performance across all tasks and information settings.
- **C-Agent: DeepSeek-R1**: low to mid-tier performance across all tasks and performance; performance comparable with that of C-Agent: Llama 4 Maverick with no statistically significant difference.

L.5 Summary

Overall, the bootstrap intervals provide a robust, model-free quantification of uncertainty, helping to pinpoint truly statistically significant performance gaps after accounting for variability across both repositories and individual bounties. The key findings are as follows:

- Code generation tasks had the highest and most stable success. In *Patch*, both OpenAI Codex CLI: o3-high and OpenAI Codex CLI: o4-mini significantly outperformed all 7 custom agents. In *Patch*, Claude Code and OpenAI Codex CLI with o3-high and o4-mini had the narrowest CIs in comparison to other agents as well as their own CIs in other task settings. However, among themselves, the performance difference is non-significant.
- Outside the advantages observed in *Patch*, the only other statistically significant leads were between OpenAI Codex CLI: o3-high vs. C-Agent: GPT-4.1, Gemini 2.5, Qwen3 235B A22B, and Llama 4 Maverick in *Detect* CWE+Title. Additionally, C-Agent: Claude 3.7 had

a significant lead over C-Agent: Qwen3 235B A22B in *Exploit*. All other pairwise agent differences were non-significant, and no model dominated across all tasks and information settings.

- *Detect* proved difficult for all agents, and success rates became more statistically significant for all agents only when CWE and Title information were provided.

M Patch Invariants

Invariants are conditions that must hold for a system to be functional. These include code invariants (e.g., unit and integration tests) and runtime invariants (e.g., server and database health checks). These invariants help prevent trivial patches (e.g., `rm -rf` the codebase to prevent all vulnerabilities). While invariants either pass or fail, we keep track of granular scores and thresholds internally to better understand the effects of any changes. To mitigate flakiness, we execute invariants three times per evaluation and take the max score. We observed that the C-Agent: Gemini 2.5 InvokeAI 0 run was the only instance that produced an inconsistent score across the three invariant test executions, with results of 435/435, 1/1, -/435, 1/1, and 435/435, 1/1, where “-” indicates the test suite failed to produce a score during the second execution.

M.1 Invariants Example: Pytorch-Lightning

For the pytorch-lightning system, we implemented runtime invariants that verify system health and responsiveness, and code invariants that ensure both frontend and core functionalities remain intact after patching. For pytorch-lightning snapshot 0, we established threshold requirements that any valid patch must satisfy: 33 passing tests for frontend functionality, 50 for core functionality, and 1 for the runtime health check. These thresholds were adjusted slightly for snapshot 1 to account for evolved functionality, with the core functionality requirement increasing to 51 tests while maintaining the same frontend and health check thresholds.

Tables 10- 16 provide overviews of all the patch runs in which the agents successfully prevented the exploit but failed invariants. In the C-Agent: Gemini 2.5 pytorch-lightning 0 run, the agent’s proposed patch for an insecure deserialization vulnerability scores: 1/1 for health check, 33/33 from frontend tests, and 43/50 for core tests 15.

The agent modified two files, `core/app.py` and `api/request_types.py`, with an updated DeepDiff Delta object implementation which broke functionality and caused seven core tests to fail, primarily in the Lightning API and application state management. The failing tests included `test_app_state_api`, `test_app_state_api_with_flows`, multiple variants of `test_start_server`, and `test_lightning_app_aggregation_speed`. Our invariant thresholds helped reveal that the patch compromised core functionality and was therefore an invalid patch.